

Chapter 1. Introduction

Recommendation systems are integral to the development of the internet that we know today and are a central function of emerging technology companies. Beyond the search ranking that opened the web's breadth to everyone, the new and exciting movies all your friends are watching, or the most relevant ads that companies pay top dollar to show you lie more applications of recommendation systems every year. The addictive For You page from TikTok, the Discover Weekly playlist by Spotify, board suggestions on Pinterest, and Apple's App Store are all hot technologies enabled by the recommendation systems. These days, sequential transformer models, multimodal representations, and graph neural nets are among the brightest areas of R&D in machine learning (ML)—all being put to use in recommendation systems.

Ubiquity of any technology often prompts questions of how the technology works, why it has become so common, and if we can get in on the action. For recommendation systems, the *how* is quite complicated. We'll need to understand the geometry of taste, and how only a little bit of interaction from a user can provide us a *GPS signal* in that abstract space. You'll see how to quickly gather a great set of candidates and how to refine them to a cohesive set of recommendations. Finally, you'll learn how to evaluate your recommender, build the endpoint that serves inference, and log about its behavior.

We will formulate variants of the core problem to be solved by recommendation systems but, ultimately, the motivating problem framing is as follows:

Given a collection of things that may be recommended, choose an ordered few for the current context and user that best match according to a certain objective.

Key Components of a Recommendation System

As we increase complexity and sophistication, let's keep in mind the components of our system. We will use *string diagrams* to keep track of our components, but in the literature these diagrams are presented in a variety of ways.

We will identify and build on three core components of recommendation systems: the collector, ranker, and server.

Collector

The collector's role is to know what is in the collection of things that may be recommended, and the necessary features or attributes of those things. Note that this collection is often a subset based on context or state.

Ranker

The ranker's role is to take the collection provided by the collector and order some or all of its elements, according to a model for the context and user.

Server

The server's role is to take the ordered subset provided by the ranker, ensure that the necessary data schema is satisfied—including essential business logic—and return the requested number of recommendations.

Take, for example, a hospitality scenario with a waiter:

When you sit down at your table, you look at the menu, unsure of what you should order. You ask the waiter, “What do you think I should order for dessert?”

The waiter checks their notes and says, “We’re out of the key lime pie, but people really like our banana cream pie. If you like pomegranate, we make pom ice cream from scratch; and it’s hard to go wrong with the donut a la mode—it’s our most popular dessert.”

In this short exchange, the waiter first serves as a collector: identifying the desserts on the menu, accommodating current inventory conditions, and preparing to talk about the characteristics of the desserts by checking their notes.

Next, the waiter serves as a ranker; they mention items high scoring in popularity (banana cream pie and donut a la mode) as well as a contextually high match item based on the patron’s features (if they like pomegranate).

Finally, the waiter serves the recommendations verbally, including both explanatory features of their algorithm and multiple choices.

While this seems a bit cartoonish, remember to ground discussions of recommendation systems in real-world applications. One of the advantages of working in RecSys is that inspiration is always nearby.

Simplest Possible Recommenders

We’ve established the components of a recommender, but to really make this practical, we need to see this in action. While much of the book is dedicated to practical recommendation systems, first we’ll start with a toy and scaffold from there.

The Trivial Recommender

The absolute simplest recommender is not very interesting but can still be demonstrated in the framework. It’s called *the trivial recommender*

(TR) because it contains virtually no logic:

```
def get_trivial_recs() -> Optional[List[str]]:
    item_id = random.randint(0, MAX_ITEM_INDEX)

    if get_availability(item_id):
        return [item_id]
    return None
```

Notice that this recommender may return either a specific `item_id` or `None`. Also observe that this recommender takes no arguments, and `MAX_ITEM_INDEX` is referencing a variable out of scope. Software principles ignored, let's think about the three components:

Collector

A random `item_id` is generated. The TR collects by checking the availability of `item_id`. We could argue that having access to `item_id` is also part of the collector's responsibility. Conditional upon the availability, the collection of recommendable things is either `[item_id]` or `None` (recall that `None` is a collection in the set-theoretic sense).

Ranker

The TR ranks with a no-op; i.e., the ranking of 1 or 0 objects in a collection is the identity function on that collection, so we merely do nothing and move on to the next step.

Server

The TR serves recommendations by its `return` statements. The only schema that's been specified in this example is that the return type is `Optional[List[str]]`.

This recommender, which is not interesting or useful, provides a skeleton that we will add to as we develop further.

Most-Popular-Item Recommender

The *most-popular-item recommender* (MPIR) is the simplest recommender that contains any utility. You probably won't want to build applications around it, but it's useful in tandem with other components in addition to providing a basis for further development.

An MPIR works just as it says; it returns the most popular items:

```
def get_item_popularities() -> Optional[Dict[str, int]]:
    ...
    # Dict of pairs: (item-identifier, count times item chosen)
    return item_choice_counts
return None

def get_most_popular_rec(max_num_recs: int) -> Optional[List[str]]:
    items_popularity_dict = get_item_popularities()
    if items_popularity_dict:
        sorted_items = sorted(
            items_popularity_dict.items(),
            key=lambda item: item[1],
            reverse=True,
        )
        return [i[0] for i in sorted_items][:max_num_recs]
    return None
```

Here we assume that `get_item_popularities` has knowledge of all available items and the number of times they've been chosen.

This recommender attempts to return the k most popular items available. While simple, this is a useful recommender that serves as a great place to start when building a recommendation system. Additionally, we will see this example return over and over, because other recommenders use this core and iteratively improve the internal components.

Let's look at the three components of our system again:

Collector

The MPIR first makes a call to `get_item_popularities` that—via database or memory access—knows which items are available and how many times they’ve been selected. For convenience, we assume that the items are returned as a dictionary, with keys given by the string that identifies the item, and values indicating the number of times that item has been chosen. We implicitly assume here that items not appearing in this list are not available.

Ranker

Here we see our first simple ranker: ranking by sorting on values. Because the collector has organized our data such that the values of the dictionary are the counts, we use the Python built-in sorting function `sorted`. Note that we use `key` to indicate that we wish to sort by the second element of the tuples—in this case, equivalent to sorting by values—and we send the `reverse` flag to make our sort descending.

Server

Finally, we need to satisfy our API schema, which is again provided via the return type hint: `Optional[List[str]]`. This wants the return type to be the nullable list of item-identifier strings that we’re recommending, so we use a list comprehension to grab the first element of the tuples. But wait! Our function has this `max_num_recs` field—what might that be doing there? Of course, this is suggesting that our API schema is looking for no greater than `max_num_recs` in the response. We handle this via the slice operator, but note that our return is between 0 and `max_num_recs` results.

Consider the possibilities at your fingertips equipped with the MPIR; recommending customers’ favorite item in each top-level category could make for a simple but useful first stab at recommendations for e-commerce. The most popular video of the day may make for a good home-page experience on your video site.

A Gentle Introduction to JAX

Since this book has *JAX* in the title, we will provide a gentle introduction to JAX here. Its official documentation can be found [on the JAX website](#).

JAX is a framework for writing mathematical code in Python that is just-in-time (JIT) compiled. JIT compilation allows the same code to run on CPUs, GPUs, and TPUs. This makes it easy to write performant code that takes advantage of the parallel-processing power of vector processors.

Additionally, one of the design philosophies of JAX is to support tensors and gradients as core concepts, making it an ideal tool for ML systems that utilize gradient-based learning on tensor-shaped data. The easiest way to play with JAX is probably via [Google Colab](#), which is a hosted Python notebook on the web.

Basic Types, Initialization, and Immutability

Let's start by learning about JAX types. We'll construct a small, three-dimensional vector in JAX and point out some differences between JAX and NumPy:

```
import jax.numpy as jnp
import numpy as np

x = jnp.array([1.0, 2.0, 3.0], dtype=jnp.float32)

print(x)
[1. 2. 3.]

print(x.shape)
(3,)

print(x[0])
1.0

x[0] = 4.0
```

```
TypeError: '<class 'jaxlib.xla_extension.ArrayImpl'>'
object does not support item assignment. JAX arrays are immutable.
```

JAX's interface is mostly similar to that of NumPy. We import JAX's version of NumPy as `jnp` to distinguish it from NumPy (`np`) by convention so that we know which version of a mathematical function we want to use. This is because sometimes we might want to run code on a vector processor like a GPU or TPU that we can use JAX for, or we might prefer to run some code on a CPU in NumPy.

The first point to notice is that JAX arrays have types. The typical float type is `float32`, which uses 32 bits to represent a floating-point number. Other types exist, such as `float64`, which has greater precision, and `float16`, which is a half-precision type that usually only runs on some GPUs.

The other point to note is that JAX tensors have shape. This is usually a tuple, so `(3,)` means a three-dimensional vector along the first axis. A matrix has two axes, and a tensor has three or more axes.

Now we come to places where JAX differs from NumPy. It is really important to pay attention to [“JAX—The Sharp Bits”](#) to understand these differences. JAX's philosophy is about speed and purity. By making functions pure (without side effects) and by making data immutable, JAX is able to make some guarantees to the underlying accelerated linear algebra (XLA) library that it uses to talk to GPUs. JAX guarantees that these functions applied to data can be run in parallel and have deterministic results without side effects, and thus XLA is able to compile these functions and make them run much faster than if they were run just on NumPy.

You can see that modifying one element in `x` results in an error. JAX would prefer that the array `x` is replaced rather than modified. One way to modify elements in an array is to do it in NumPy rather than JAX and convert NumPy arrays to JAX—for example, using `jnp.array(np_array)`—when the subsequent code needs to run fast on immutable data.

Indexing and Slicing

Another important skill to learn is that of indexing and slicing arrays:

```
x = jnp.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]], dtype=jnp.int32)

# Print the whole matrix.
print(x)
[[1 2 3]
 [4 5 6]
 [7 8 9]]

# Print the first row.
print(x[0])
[1 2 3]

# Print the last row.
print(x[-1])
[7 8 9]

# Print the second column.
print(x[:, 1])
[2 5 8]

# Print every other element
print(x[::2, ::2])
[[1 3]
 [7 9]]
```

NumPy introduced indexing and slicing operations that allow us to access different parts of an array. In general, the notation follows a `start:end:stride` convention. The first element indicates where to start, the second indicates where to end (but not inclusive), and the stride indicates the number of elements to skip over. The syntax is similar to that of the Python `range` function.

Slicing allows us to access views of a tensor elegantly. Slicing and indexing are important skills to master, especially when we start to manipulate

tensors in batches, which we typically do to make the most use of acceleration hardware.

Broadcasting

Broadcasting is another feature of NumPy and JAX to be aware of. When a binary operation such as addition or multiplication is applied to two tensors of different sizes, the tensor with axes of size 1 is lifted up in rank to match that of the larger-sized tensor. For example, if a tensor of shape $(3,3)$ is multiplied by a tensor of shape $(3,1)$, the rows of the second tensor are duplicated before the operation so that it looks like a tensor of shape $(3,3)$:

```
x = jnp.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]], dtype=jnp.int32)

# Scalar broadcasting.
y = 2 * x
print(y)
[[ 2  4  6]
 [ 8 10 12]
 [14 16 18]]

# Vector broadcasting. Axes with shape 1 are duplicated.
vec = jnp.reshape(jnp.array([0.5, 1.0, 2.0]), [3, 1])
y = vec * x
print(y)
[[ 0.5  1.   1.5]
 [ 4.   5.   6. ]
 [14.  16.  18. ]]

vec = jnp.reshape(vec, [1, 3])
y = vec * x
print(y)
[[ 0.5  2.   6. ]
 [ 2.   5.  12. ]
 [ 3.5  8.  18. ]]
```

The first case is the simplest, that of scalar multiplication. The scalar is multiplied throughout the matrix. In the second case, we have a vector of

shape `(3,1)` multiplying the matrix. The first row is multiplied by 0.5, the second row is multiplied by 1.0, and the third row is multiplied by 2.0. However, if the vector has been reshaped to `(1,3)`, the columns are multiplied by the successive entries of the vector instead.

Random Numbers

Along with JAX's philosophy of pure functions comes its particular way of handling random numbers. Because pure functions do not cause side effects, a random-number generator cannot modify the random number seed, unlike other random-number generators. Instead, JAX deals with random-number keys whose state is updated explicitly:

```
import jax.random as random

key = random.PRNGKey(0)
x = random.uniform(key, shape=[3, 3])
print(x)
[[0.35490513 0.60419905 0.4275843 ]
 [0.23061597 0.6735498  0.43953657]
 [0.25099766 0.27730572 0.7678207 ]]

key, subkey = random.split(key)
x = random.uniform(key, shape=[3, 3])
print(x)
[[0.0045197  0.5135027  0.8613342 ]
 [0.06939673 0.93825936 0.85599923]
 [0.706004   0.50679076 0.6072922 ]]

y = random.uniform(subkey, shape=[3, 3])
print(y)
[[0.34896135 0.48210478 0.02053976]
 [0.53161216 0.48158717 0.78698325]
 [0.07476437 0.04522789 0.3543167  ]]
```

JAX first requires you to create a random-number `key` from a seed. This key is then passed into random-number generation functions like `uniform` to create random numbers in the 0 to 1 range.

To create more random numbers, however, JAX requires that you split the key into two parts: a new key to generate other keys, and a subkey to generate new random numbers. This allows JAX to deterministically and reliably reproduce random numbers even when many parallel operations are calling the random-number generator. We just split a key into as many parallel operations as needed, and the random numbers resulting are now randomly distributed but also reproducible. This is a nice property when you want to reproduce experiments reliably.

Just-in-Time Compilation

JAX starts to diverge from NumPy in terms of execution speed when we start using JIT compilation. JITing code—transforming the code to be compiled just in time—allows the same code to run on CPUs, GPUs, or TPUs:

```
import jax

x = random.uniform(key, shape=[2048, 2048]) - 0.5

def my_function(x):
    x = x @ x
    return jnp.maximum(0.0, x)

%timeit my_function(x).block_until_ready()
302 ms ± 9 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

my_function_jitted = jax.jit(my_function)

%timeit my_function_jitted(x).block_until_ready()
294 ms ± 5.45 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

The JITed code is not that much faster on a CPU but will be dramatically faster on a GPU or TPU backend. Compilation also carries some overhead when the function is called the first time, which can skew the timing of the first call. Functions that can be JITed have restrictions, such as mostly calling JAX operations inside and having restrictions on loop operations. Variable-length loops trigger frequent recompilations. The [“Just-in-Time](#)

[Compilation with JAX” documentation](#) covers a lot of the nuances of getting functions to JIT compile.

Summary

While we haven’t done much math yet, we have gotten to the point where we can begin providing recommendations and implementing deeper logic into these components. We’ll start doing things that look like ML soon enough.

So far, we have defined what a recommendation problem is, set up the core architecture of our recommendation system—the collector, the ranker, and the server—and shown a couple of trivial recommenders to illustrate how the pieces come together.

Next we’ll explain the core relationship that recommendation systems seek to exploit: the user-item matrix. This matrix lets us build a model of personalization that will lead to ranking.